

A06

INSECURE DESIGN

Design flaws · missing threat model
absent security requirements · business logic failures

A10

MISHANDLING EXCEPTIONAL CONDITIONS

Stack trace disclosure · fail-open logic
unhandled exceptions · error-based data leakage

COURSE POSITION: These two categories complete the design strand of the course. A06 flaws cannot be fixed by patching code — they require a redesign. A10 is new in OWASP Top 10:2025 (SSRF absorbed into A05); it addresses what happens when your design ignores error conditions. Cross-references: Module 5 (A01 access control design) · Module 6 (A05 injection design) · Module 7 (SSDLC threat modelling) · Module 9 (A04/A08 crypto & integrity design) · Module 11 (A09 logging — what to log when exceptions occur).

Activity	Focus	Primary Tool	Time
Activity 1	Exception Disclosure — Juice Shop error responses	Juice Shop · ZAP Proxy	10 min
Activity 2	Business Logic Design Flaws — Admin Panel & Feedback	Juice Shop · ZAP	15 min
Activity 3	Mini STRIDE Threat Model — new feature walkthrough	Pen & paper / group	12 min
Activity 4	Fail-Open vs Fail-Closed — DVWA error-based SQLi	DVWA · Firefox DevTools	8 min
Activity 5	Defence Mapping & Debrief	Group discussion	5 min

ACTIVITY 1 — Exception Disclosure: Forcing Juice Shop to Reveal Its Internals 🕒 10 min

A10 Mishandling of Exceptional Conditions · Juice Shop · ZAP Proxy

WHY IT MATTERS: A10 is brand-new in OWASP Top 10:2025. It covers what happens when a web application encounters an unexpected condition — a malformed request, a missing resource, a database error — and responds with information it should never reveal. Stack traces disclose internal paths, library versions, and database schema. Exception messages can confirm whether a username exists. Both represent design failures: a properly designed application should produce safe, generic error responses in all error paths — not just the happy path.

Step-by-step

1. Open Firefox and ensure ZAP Proxy is intercepting on `localhost:8080`
2. In Juice Shop, navigate to the login page. Enter a valid username but an intentionally malformed password containing special characters: `admin@juice-sh.op' OR 1=1--`
3. In ZAP History, find the POST request to `/rest/user/login`. Examine the full response body. Does it contain a stack trace, framework version string, or database error message? Note anything that reveals internal implementation details.
4. Now deliberately send a malformed API request using ZAP's Request Editor. Edit any GET request to a `/api/` endpoint and change the numeric ID to a non-numeric string, e.g.: `GET /api/Users/abc123`
5. Examine the 500 or 400 response. How verbose is the error? Does it name the ORM, the database, or internal file paths?

6. Navigate to a non-existent Juice Shop page, e.g.:
<http://localhost:3000/api/NonExistentEndpoint/99999>
7. Compare the error response format. Is it consistent with other error responses? Does it reveal different information depending on the error type?
8. Now visit a legitimate page: try causing a 404 on a well-secured site such as
<https://bbc.co.uk/nonexistent>
9. Compare BBC's error response to Juice Shop's. What does a production-grade safe error response look like?

EXPLORER	Identify at least two different error responses from Juice Shop that reveal internal implementation details. Write down exactly what information is disclosed and which OWASP A10 sub-category it falls into (stack trace, framework disclosure, or database error).
PRACTITIONER	Force three distinct error types in Juice Shop (4xx, 5xx, and an ORM error). For each, write a one-sentence 'safe' replacement response that provides no useful information to an attacker while still being useful to a legitimate developer debugging the issue.
EXPERT	Write a Flask error handler (using <code>@app.errorhandler</code>) that intercepts all exceptions, logs the full stack trace server-side, and returns only a generic JSON response to the client. Add a correlation ID to both the log entry and the response so support teams can match client errors to server logs without exposing internals. Reference: Module 11 (A09 Logging) — your handler connects directly to that module's logging requirements.

ACTIVITY 2 — Business Logic Design Flaws in Juice Shop 15 min

A06 Insecure Design · Admin Panel, Forged Feedback & Zero-Star Review · ZAP

WHY IT MATTERS: A06 Insecure Design covers flaws that exist because a security decision was never made at design time — not because code was written badly. These cannot be fixed with a patch. The three scenarios below are design failures: the admin panel was never designed to require a role check; the feedback form was designed to accept a user-supplied UserId; the star rating was only enforced in the UI, never in the API contract. Compare: Module 5 (A01) showed how to enforce access control — these scenarios show what happens when the access control requirement was absent from the design entirely.

Part A — Admin Panel Access (5 min)

10. Log into Juice Shop as a regular user (not admin). In the browser URL bar, navigate directly to:
<http://localhost:3000/#/administration>
11. What do you see? The administration panel — listing all users, all feedback, all orders — is visible to any authenticated user with no role check. Attempt to delete a feedback entry.
12. In ZAP History, identify the API call that deletes the feedback. What does the request contain that authorises the deletion? Is there any role verification in the request or response?
13. Explain: is this an implementation bug or a design flaw? Could Bandit SAST (Module 7) have detected this? Why not?

Part B — Forged Feedback: User-Controlled UserId (5 min)

14. In Juice Shop, go to the Customer Feedback form. Write some feedback and submit it. Observe the POST request in ZAP.
15. In ZAP's Request Editor, resend the POST to `/api/Feedbacks` with the `UserId` field changed to `1`
16. Check the administration panel. The feedback now appears attributed to user ID 1 (`admin@juice-sh.op`). The server accepted a user-supplied identity claim without verification.
17. Recall the ShopCo SSDLC analysis from Module 7. At which SSDLC phase (Requirements, Design, Development, SAST, Test/DAST) should this design decision have been challenged? Write your answer and be ready to defend it.

Part C — Zero-Star Rating: UI Enforcement vs API Contract (5 min)

18. The Juice Shop star-rating widget prevents selecting zero stars in the browser UI. In ZAP, intercept a feedback submission and change the 'rating' field value to `0`
19. Does the server reject it? A properly designed API contract specifies the valid range of every field at the API layer, not only in the UI. This is a design failure: the constraint existed in the presentation layer but was never carried through to the data contract.
20. Repeat with a value of `999`
21. What does the response show? Write a one-sentence server-side validation rule that would prevent both zero and out-of-range values.

EXPLORER	Complete Part A only. Navigate to the admin panel as a non-admin user and note what is visible. Explain in one sentence why role-based access control must be enforced at the API layer, not only in the navigation menu.
PRACTITIONER	Complete all three parts. For Part B, explain the difference between authentication ('who are you?') and authorisation ('what can you do?') — and identify which one is missing from the UserId design. Connect this to the access control design principle from Module 5.
EXPERT	After completing all parts, write a short threat model entry (one row of a STRIDE table) for the feedback submission feature. Use the format: Threat category What an attacker could do Design control that prevents it SSDLC phase. Then connect your entry to the STRIDE exercise in Activity 3.

ACTIVITY 3 — Mini STRIDE Threat Model: Designing a New Feature Securely

🕒 12 min

A06 Insecure Design · STRIDE · Connects to Module 7 SSDLC · Group or pairs

SCENARIO: Juice Shop is adding a new 'Gift Card' feature. Users will be able to: (1) purchase a gift card with a monetary value; (2) receive a unique code via email; (3) redeem the code at checkout for a discount equal to the card's value; (4) the redeemed card is then marked as used and cannot be redeemed again. Your task: apply STRIDE to this new feature BEFORE any code is written — exactly as you would in the Requirements phase of the SSDLC (Module 7).

The STRIDE Framework (quick recap from Module 7)

S	T	Reference
S	Spoofing	Attacker claims to be another user during gift card redemption — uses someone else's code before they do
T	Tampering	Attacker intercepts the redemption request and changes the card value from £10 to £10,000
R	Repudiation	User claims they never redeemed their card; no log entry proves they did
I	Info Disclosure	Gift card codes are predictable sequential numbers — attacker enumerates valid codes
D	Denial of Service	Attacker calls the redemption endpoint 10,000 times per second, crashing the service
E	Elevation of Privilege	Redemption endpoint does not check the logged-in user's identity — any authenticated user can redeem any code

Your task: for each STRIDE threat, write the design control that prevents it. Use the table below.

	Threat	Gift Card Scenario: What the Attacker Does	Design Control (write your answer here)
S	Spoofing	Attacker redeems code belonging to another user	
T	Tampering	Intercepts redemption request, modifies card value	
R	Repudiation	User denies redemption; no audit log exists	
I	Info Disclosure	Sequential card codes enumerable by attacker	
D	DoS	Hammers redemption endpoint to exhaust server	
E	Elev. of Privilege	Redemption succeeds without session check	

ANSWER GUIDE: S → Authenticate ownership: code is bound to the purchasing user's account, redeemable only when that user is logged in. T → Server derives the discount from the stored card record, never from the request body (A08 integrity pattern from Module 9). R → Log every redemption attempt (success and failure) with user_id, code_hash, timestamp, correlation_id — connects forward to Module 11 A09. I → Use cryptographically random codes (min 128-bit entropy) — not sequential IDs (A04 pattern from Module 9). D → Rate-limit the redemption endpoint: max 5 attempts per user per hour (A06 rate-limit design principle). E → Verify session user_id == card.owner_id before accepting redemption — the same access control pattern from Module 5.

ACTIVITY 4 — Fail-Open vs Fail-Closed: DVWA Error-Based SQLi ⌚ 8 min

A10 Mishandling of Exceptional Conditions · DVWA · Firefox DevTools

WHY IT MATTERS: Error-based SQL injection is a direct consequence of A10: the database raises an exception when it encounters malformed SQL, and the application forwards that exception to the browser unfiltered. This gives the attacker a read channel into the database schema without needing a UNION or blind technique. The design question is: when the database raises an error, what should the application do? Fail open (pass the error to the client) or fail closed (log it server-side, return a generic response)?

Step-by-step

- Open Firefox and navigate to DVWA: <http://localhost/dvwa>
- Log in (admin / password). Set Security Level to Low. Go to SQL Injection.
- In the User ID field, enter a single quote to cause a database syntax error: `'`
- Read the error response. It contains the full MySQL error message, the query structure, and the database name. Record what is disclosed.
- Now extract the database version using an error-based payload: `1 AND EXTRACTVALUE(1, CONCAT(0x7e, VERSION()))`
- The EXTRACTVALUE function raises an XPath error containing the version string. Record the exact version. How would an attacker use this information?
- Extract the current database user: `1 AND EXTRACTVALUE(1, CONCAT(0x7e, USER()))`
- Set Security Level to Medium. The application now uses parameterised queries. Repeat the same payloads. What do you observe? The error is now handled server-side and a generic message is returned to the client.

30. Set Security Level to High. Observe that the input field is replaced with a dropdown. What design decision eliminated the attack surface entirely?

EXPLORER	Complete steps 1–4. Record the information disclosed by the database error on Low security. In one sentence, explain why forwarding database exceptions to the browser is an A10 failure.
PRACTITIONER	Complete all steps through to Medium security. Compare the Low and Medium error responses. Explain why parameterised queries (A05 Module 6 defence) simultaneously prevent SQL injection AND prevent error-based information disclosure.
EXPERT	On High security, the input becomes a dropdown. Write a one-paragraph design note explaining: (a) why reducing the attack surface by design is more robust than input filtering; (b) how this connects to the principle of 'the server owns the truth' from Module 9 (A08); and (c) what error logging pattern (Module 11 A09) should accompany this to ensure developers can still debug without exposing internals.

ACTIVITY 5 — Defence Mapping & Debrief ⌚ 5 min

Group — connect each attack to its design principle

Attack / Finding	OWASP Category	Root Design Failure	Design Principle That Prevents It
Stack trace in Juice Shop response	A10	Exception handler passes error to client	Fail closed: log server-side, return generic response
Admin panel accessible to all users	A06	Role requirement never designed in	Complete mediation: check every access, every time
Forged feedback UserId	A06	User identity accepted from request, not session	Server owns the truth: derive identity server-side
Zero-star / out-of-range rating	A06	API contract never enforced server-side	Validate at the API boundary, not only in the UI
Error-based SQLi data extraction	A10	Database errors returned to browser	Parameterise queries + handle exceptions internally
STRIDE threat I: enumerable card codes	A06	Sequential ID used where random code needed	Use cryptographically random values (Module 9, A04)

CONNECTING FORWARD: Module 11 (A09 Logging & Alerting Failures) picks up directly from Activities 1 and 4. The key question for tomorrow: once you have replaced verbose error responses with generic ones, how do you ensure developers and security teams can still diagnose real incidents? The answer is structured server-side logging with correlation IDs — exactly what Activity 1's Expert challenge asked you to design.

Further Reading & Resources

- OWASP Top 10:2025 — A06 Insecure Design: owasp.org/Top10/A06_2021-Insecure_Design/ (2021 content, 2025 renumbered)
- OWASP Top 10:2025 — A10 Mishandling of Exceptional Conditions: new in 2025 RC — see Module 4 Transition Card for overview
- OWASP ASVS V11 (Business Logic) and V13 (API): asvs.owasp.org — design-time checklist for both A06 and A10
- Microsoft STRIDE Threat Modelling: docs.microsoft.com/en-us/azure/security/develop/threat-modeling-tool
- OWASP Threat Dragon (free threat modelling tool): owasp.org/www-project-threat-dragon
- Juice Shop CTF — Business Logic challenges: pwning.owasp-juice.shop/part2/improper-input-validation

- PortSwigger Web Security Academy — Business Logic Vulnerabilities: portswigger.net/web-security/logic-flaws